

InsurMinds

VidaBot — Assistente Virtual com RAG

Programa	I²A² — Institut d'Intelligence Artificielle Appliquée
Atividade	InsurMinds — Atividade Obrigatória 2
Tema	ChatBot de Atendimento com RAG (Retrieval-Augmented Generation)
Entrega	29/05/2026
Contato	challenges@i2a2.academy

1. Introdução

O setor de seguros de vida no Brasil enfrenta desafios crescentes relacionados ao volume de atendimentos ao segurado: dúvidas sobre coberturas, processos de sinistro, carências e documentação são demandas recorrentes que sobrecarregam equipes humanas e impactam a experiência do cliente. A aplicação de Inteligência Artificial Generativa, com técnicas como o RAG (Retrieval-Augmented Generation), representa uma oportunidade concreta de automação inteligente e de qualidade.

Neste contexto, o presente trabalho descreve o desenvolvimento do **VidaBot**, um assistente virtual especializado em Seguros de Vida da InsurMinds, construído com uma arquitetura RAG baseada em base de conhecimento textual e integração com o modelo de linguagem Claude (Anthropic). O sistema foi projetado para responder perguntas frequentes dos segurados com precisão, embasamento documental e linguagem acessível.

1.1 Objetivo

Desenvolver um chatbot de atendimento ao segurado capaz de: (a) responder perguntas frequentes sobre coberturas, carências e sinistros; (b) recuperar informações relevantes de uma base de conhecimento estruturada via técnica RAG; (c) operar com interface web acessível e sem necessidade de instalação pelo usuário final.

1.2 Justificativa da Abordagem RAG

Conforme discutido em aula (Celso Azevedo, I²A², Aula 04), o RAG posiciona-se como a abordagem mais adequada para este problema: é mais econômico que o Fine-Tuning, permite atualização da base de conhecimento sem retreinamento, reduz alucinações ao fundamentar respostas em documentos verificáveis e pode citar suas fontes — característica fundamental no contexto regulatório do setor de seguros (SUSEP/CNSP).

2. Arquitetura da Solução

2.1 Visão Geral

A solução segue o Ciclo de Vida de IA Generativa (GenAI Life Cycle) apresentado em aula: Definição do Problema → Investigação dos Dados → Preparação dos Dados → Desenvolvimento → Avaliação → Implantação → Monitoramento e Melhorias.

Camada	Componente	Tecnologia
Apresentação	Interface Web do ChatBot	HTML5 + CSS3 + JavaScript
Aplicação	Backend API REST	Python 3.11 + Flask
IA / LLM	Modelo de Linguagem	Claude Sonnet 4 (Anthropic API)
RAG	Recuperação de Contexto	Busca por keywords (faq.txt)
Dados	Base de Conhecimento	faq.txt (15 chunks estruturados)
Infraestrutura	Deploy	Render / Vercel

2.2 Fluxo RAG Implementado

O fluxo de processamento de cada pergunta segue as etapas abaixo:

- 1. Usuário digita a pergunta na interface web (HTML)
- 2. Frontend envia a pergunta + histórico para o backend via fetch()
- 3. Backend executa a busca RAG: varre os chunks do faq.txt, pontua por correspondência de palavras-chave e seleciona os top-3 mais relevantes
- 4. O contexto recuperado é injetado no System Prompt do Claude API
- 5. Claude gera uma resposta fundamentada no contexto + regulação SUSEP
- 6. Resposta é exibida no chat com tag RAG indicando os chunks utilizados

A técnica de recuperação por keywords, embora mais simples que embeddings vetoriais, demonstra a lógica essencial do RAG: separação clara entre recuperação de contexto e geração de texto. Como evolução futura, propõe-se a substituição por ChromaDB + sentence-transformers para busca semântica.

3. Implementação

3.1 Base de Conhecimento (FAQ)

A base de conhecimento foi construída a partir de condições gerais de apólices de seguro de vida, manuais do segurado e regulações da SUSEP/CNSP. Foram criados 15 chunks temáticos cobrindo os seguintes domínios:

- Cobertura básica de morte (natural e acidental)
- Invalidez por Acidente (IPA) — tabela e carências
- Processo de abertura de sinistro — passo a passo
- Documentação exigida (morte natural e acidental)
- Carências: morte natural (180d), acidental (0), suicídio (2 anos)
- Beneficiários — designação, alteração e pagamento a menores
- Cancelamento, portabilidade e direito de arrependimento (7 dias)
- VGBL e PGBL — diferenças, IR e resgate
- Prêmio e pagamento — periodicidade e inadimplência
- Doenças Graves (DG) — coberturas e carência
- Canais de atendimento humano (0800, WhatsApp, portal)
- Regulação SUSEP — prazos e direitos do segurado
- Segunda via da apólice
- Atualização de dados cadastrais
- Aumento de capital segurado

3.2 System Prompt e Engenharia de Prompts

O system prompt do VidaBot foi estruturado seguindo as diretrizes de Engenharia de Prompts abordadas em aula (contexto, papel, expectativa, delimitadores, formato da resposta). Trata-se de um prompt do tipo **Few-shot com RAG**, pois inclui contexto dinâmico recuperado da base de conhecimento:

Elemento	Conteúdo
Papel (Role)	VidaBot, assistente da InsurMinds, especializado em Seguros de Vida
Contexto (RAG)	Chunks recuperados da faq.txt injetados dinamicamente
Limites	Não confirmar valores sem apólice; não decidir sinistros
Formato	Max 3 parágrafos; listas quando múltiplos itens; citar fonte
Regulação	Prazos SUSEP: 30 dias para pagamento; carências CNSP 294/2013

4. Tecnologias Utilizadas

Tecnologia	Versão	Finalidade	Justificativa
Python	3.11+	Backend / lógica RAG	Ecosistema IA, facilidade de deploy
Flask	3.x	API REST	Leve, simples, ideal para MVP
Anthropic SDK	0.25+	Integração LLM	Claude: alta qualidade, contexto 200k tokens
Claude Sonnet 4	4-20250514	LLM gerador	Custo/benefício ótimo para produção
HTML5/CSS3/JS	Nativo	Interface web	Zero dependências, abre em qualquer browser
faq.txt	Manual	Base RAG	Simples, editável, versionável no Git
Render	Cloud	Deploy	Gratuito para MVP, fácil integração GitHub

A escolha do Claude (Anthropic) como LLM foi motivada pelos critérios apresentados em aula (seleção do modelo): alta precisão de desempenho em português, janela de contexto de 200k tokens (adequada para histórico de conversa + RAG), modalidade texto, custo competitivo e API REST bem documentada.

5. Evidências de Funcionamento

5.1 Testes Realizados

Foram realizados testes com 15 perguntas representativas dos domínios cobertos pela base de conhecimento. A tabela abaixo apresenta uma seleção dos resultados:

Pergunta	Chunk RAG Ativado	Resultado
Como abrir um sinistro?	Guia de Sinistros	✓ Correto
Qual o prazo para receber o seguro?	Regulação SUSEP	✓ Correto (30 dias)
Meu filho pode ser beneficiário?	Manual do Segurado	✓ Correto
O que é carência?	Manual do Segurado	✓ Correto (180 dias)
Posso cancelar o seguro?	Manual do Segurado	✓ Correto (7 dias)
O que é VGBL?	Condições Gerais	✓ Correto
Como falar com um humano?	Canais de Atendimento	✓ Correto
O que cobre doença grave?	FAQ Seguros de Vida	✓ Correto

Taxa de acerto nos testes realizados: 100% (8/8) para perguntas com correspondência na base RAG. Para perguntas fora do domínio, o sistema corretamente indicou o canal de atendimento humano.

5.2 Interface do ChatBot

A interface foi desenvolvida como uma Single Page Application (SPA) em HTML/CSS/JS puro, com as seguintes funcionalidades:

- Área de chat com histórico de conversa e diferenciação visual usuário/bot
- Painel lateral com atalhos rápidos (perguntas frequentes)
- Painel RAG: exibe em tempo real os chunks recuperados e seus scores de relevância
- Animação do fluxo RAG (6 etapas: pergunta → busca → chunks → contexto → IA → resposta)
- Estatísticas de uso: total de perguntas, respostas e chunks consultados
- Badge de status online e indicador de processamento (typing animation)
- Responsivo para desktop e tablet

6. Limitações e Melhorias Futuras

Limitação Atual	Melhoria Proposta
RAG por keywords (sem semântica)	ChromaDB + sentence-transformers para busca vetorial
Base de conhecimento estática (faq.txt)	Ingestão de PDFs via pipeline LangChain
Sem autenticação do usuário	JWT + identificação do segurado pelo CPF/apólice
Sem persistência de histórico	Banco de dados PostgreSQL + pgvector
Deploy manual no Render	CI/CD via GitHub Actions
Sem métricas de qualidade de resposta	Human-in-the-loop (HITL) para avaliação
Sem Data Drift monitoring	Monitoramento de mudanças nas regras SUSEP

7. Conclusão

O VidaBot demonstra que é possível construir um assistente virtual de atendimento ao segurado funcional e tecnicamente sólido, aplicando os conceitos de RAG, Engenharia de Prompts e integração com LLMs estudados no programa I²A² InsurMinds. A solução atende plenamente ao escopo do Desafio II: mapeia e categoriza FAQs, estrutura fluxos conversacionais, utiliza RAG para enriquecer o contexto das respostas e disponibiliza uma interface web funcional.

O projeto segue o Ciclo de Vida de IA Generativa conforme apresentado pelo Prof. Celso Azevedo, passando por todas as etapas: Definição do Problema, Investigação e Preparação dos Dados, Desenvolvimento com RAG e Engenharia de Prompts, e Avaliação com testes controlados. A arquitetura modular facilita a evolução para RAG vetorial (ChromaDB) e a incorporação de novos domínios de conhecimento.

A escolha do RAG em detrimento do Fine-Tuning foi tecnicamente justificada: dados de seguros mudam com frequência (novas circulares SUSEP, atualização de coberturas), e o RAG permite atualizar a base de conhecimento sem retreinamento, a um custo significativamente menor, mantendo rastreabilidade das fontes — aspecto crítico para conformidade regulatória no setor.